

PRACTICA 6 | TALLER 1

Juan José Álvarez Restrepo

María Alexandra Jiménez Suarez

Universidad Pontificia Bolivariana, Facultad de Ingeniería

Sistemas Operativos

Fabio Andres Guzman Figueroa

25 de febrero de 2026

1. Introducción

En los sistemas operativos modernos, múltiples procesos e hilos se ejecutan de manera concurrente y comparten recursos del sistema como memoria, archivos, impresoras o dispositivos de entrada y salida. Cuando varios hilos intentan acceder simultáneamente a un mismo recurso compartido, pueden producirse conflictos conocidos como condiciones de carrera, lo que genera comportamientos inesperados.

Para resolver estos problemas se utilizan mecanismos de sincronización, entre ellos los semáforos, introducidos por el científico de la computación Edsger Dijkstra.

El presente trabajo tiene como objetivo explicar el funcionamiento de los semáforos y aplicar este concepto a la solución del clásico problema de sincronización conocido como el Problema de los Filósofos Comensales.

2. Sincronización de Hilos

La sincronización es el mecanismo que permite coordinar la ejecución de múltiples hilos para evitar inconsistencias cuando comparten recursos.

Los principales problemas que pueden presentarse son:

- Condición de carrera
- Interbloqueo (Deadlock)
- Inanición (Starvation)

3. Semáforos

Un semáforo es una variable entera protegida que controla el acceso a recursos compartidos mediante dos operaciones atómicas:

- `wait()` → Disminuye el valor del semáforo.
Si el valor es 0, el hilo queda bloqueado.
- `post()` → Incrementa el valor del semáforo y desbloquea hilos en espera.

Existen dos tipos de semáforos:

- Semáforo binario (0 o 1)
- Semáforo contador (permite varios accesos simultáneos)

Operación	Acción
wait (P)	Decrementa el contador. Si es 0, bloquea el hilo.
Post (v)	Incrementa el contador. Desbloquea un hilo en espera.

Componentes de la solución:

- **tenedor[5]:** Array de 5 semáforos, uno por tenedor. Cada semáforo inicializado en 1 (libre).
- **acceso:** Semáforo adicional inicializado en 4. Evita que los 5 filósofos intenten comer al mismo tiempo, previniendo el deadlock.
- **5 hilos (pthreads):** Uno por filósofo, ejecutándose en paralelo.

PSEUDOCODIGO:

```

VARIABLES GLOBALES:
    sem_t tenedor[5]    // Un semáforo por tenedor (iniciado en 1 = libre)
    sem_t acceso        // Semáforo para limitar acceso simultáneo (iniciado en 4)

FUNCIÓN filosofo(id):
    izq = id
    der = (id + 1) % 5

    REPETIR SIEMPRE:
        // --- PENSAR ---
        imprimir "Filósofo [id] está PENSANDO"
        esperar(tiempo_aleatorio)           // simula pensar (1-3 seg)

        // --- INTENTAR COMER ---
        wait(acceso)                        // Pide permiso al semáforo de acceso
                                           // (máx 4 filósofos intentando a la vez)

        wait(tenedor[izq])                  // Toma el tenedor izquierdo
        imprimir "Filósofo [id] tomó tenedor IZQUIERDO [izq]"

        wait(tenedor[der])                  // Toma el tenedor derecho
        imprimir "Filósofo [id] tomó tenedor DERECHO [der]"

        // --- COMER ---
        imprimir "Filósofo [id] está COMIENDO"
        esperar(tiempo_aleatorio)           // simula comer (1-2 seg)

        // --- SOLTAR TENEDORES ---
        post(tenedor[izq])                  // Libera tenedor izquierdo
        post(tenedor[der])                  // Libera tenedor derecho
        post(acceso)                        // Libera el permiso de acceso

        imprimir "Filósofo [id] terminó de comer"

```

FUNCIÓN principal:

Inicializar `tenedor[i] = 1` para `i` en `0..4`

Inicializar `acceso = 4`

Crear 5 hilos, cada uno ejecuta `filosofo(i)`

Esperar a que terminen todos los hilos (`join`)

Destruir semáforos

¿Por qué funciona?

Al restringir a 4 el número de filósofos que pueden intentar comer simultáneamente (de los 5 posibles), siempre habrá al menos un tenedor libre. Por tanto, al menos un filósofo siempre podrá tomar ambos tenedores y avanzar, eliminando el deadlock.

CODIGO

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N_FILOSOFOS 5
#define N_CICLOS 3

sem_t tenedor[N_FILOSOFOS];
sem_t acceso;

void pensar(int id) {
    printf("Filosofo %d -> PENSANDO...\n", id);
    sleep((rand() % 3) + 1);
}

void comer(int id) {
    printf("Filosofo %d -> COMIENDO [tenedores %d y %d]\n",
        id, id, (id + 1) % N_FILOSOFOS);
    sleep((rand() % 2) + 1);
}

void *filosofo(void *arg) {
    int id = *(int *)arg;
    int izq = id;
    int der = (id + 1) % N_FILOSOFOS;
```

```

    for (int c = 0; c < N_CICLOS; c++) {
        pensar(id);

        sem_wait(&acceso);

        sem_wait(&tenedor[izq]);
        printf("Filosofo %d -> Tomo tenedor IZQUIERDO (%d)\n", id, izq);

        sem_wait(&tenedor[der]);
        printf("Filosofo %d -> Tomo tenedor DERECHO (%d)\n", id, der);

        comer(id);

        sem_post(&tenedor[izq]);
        sem_post(&tenedor[der]);
        sem_post(&acceso);

        printf("Filosofo %d -> Solto tenedores.\n\n", id);
    }

    printf("==> Filosofo %d TERMINO sus %d ciclos.\n", id, N_CICLOS);
    return NULL;
}

int main(void) {
    pthread_t hilos[N_FILOSOFOS];
    int ids[N_FILOSOFOS];

    for (int i = 0; i < N_FILOSOFOS; i++)
        sem_init(&tenedor[i], 0, 1);

    sem_init(&acceso, 0, N_FILOSOFOS - 1);

    printf("=== Problema de los Filósofos Comensales ===\n");
    printf("Filósofos: %d | Ciclos por filósofo: %d\n\n", N_FILOSOFOS,
N_CICLOS);

    for (int i = 0; i < N_FILOSOFOS; i++) {
        ids[i] = i;
        pthread_create(&hilos[i], NULL, filosofo, &ids[i]);
    }

    for (int i = 0; i < N_FILOSOFOS; i++)
        pthread_join(hilos[i], NULL);
}

```

```

    for (int i = 0; i < N_FILOSOFOS; i++)
        sem_destroy(&tenedor[i]);
    sem_destroy(&acceso);

    printf("\n=== Todos los filosofos comieron. Fin del programa. ===\n");
    return 0;
}

```

Al ejecutar:

```

=== Problema de los Filósofos Comensales ===
Filosofos: 5 | Ciclos por filósofo: 3

Filosofo 0 -> PENSANDO...
Filosofo 1 -> PENSANDO...
Filosofo 2 -> PENSANDO...
Filosofo 3 -> PENSANDO...
Filosofo 4 -> PENSANDO...
Filosofo 2 -> Tomo tenedor IZQUIERDO (2)
Filosofo 2 -> Tomo tenedor DERECHO (3)
Filosofo 2 -> COMIENDO [tenedores 2 y 3]
Filosofo 0 -> Tomo tenedor IZQUIERDO (0)
Filosofo 0 -> Tomo tenedor DERECHO (1)
Filosofo 0 -> COMIENDO [tenedores 0 y 1]
Filosofo 0 -> Solto tenedores.

Filosofo 0 -> PENSANDO...
Filosofo 1 -> Tomo tenedor IZQUIERDO (1)
Filosofo 4 -> Tomo tenedor IZQUIERDO (4)
Filosofo 3 -> Tomo tenedor IZQUIERDO (3)
Filosofo 2 -> Solto tenedores.

Filosofo 2 -> PENSANDO...
Filosofo 1 -> Tomo tenedor DERECHO (2)
Filosofo 1 -> COMIENDO [tenedores 1 y 2]
Filosofo 4 -> Tomo tenedor DERECHO (0)
Filosofo 4 -> COMIENDO [tenedores 4 y 0]
Filosofo 4 -> Solto tenedores.

Filosofo 4 -> PENSANDO...
Filosofo 0 -> Tomo tenedor IZQUIERDO (0)
Filosofo 3 -> Tomo tenedor DERECHO (4)

```

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

Filosofo 4 -> PENSANDO...
Filosofo 0 -> Tomo tenedor IZQUIERDO (0)
Filosofo 3 -> Tomo tenedor DERECHO (4)
Filosofo 3 -> COMIENDO [tenedores 3 y 4]
Filosofo 1 -> Solto tenedores.

Filosofo 1 -> PENSANDO...
Filosofo 0 -> Tomo tenedor DERECHO (1)
Filosofo 0 -> COMIENDO [tenedores 0 y 1]
Filosofo 2 -> Tomo tenedor IZQUIERDO (2)
Filosofo 3 -> Solto tenedores.

Filosofo 3 -> PENSANDO...
Filosofo 2 -> Tomo tenedor DERECHO (3)
Filosofo 2 -> COMIENDO [tenedores 2 y 3]
Filosofo 4 -> Tomo tenedor IZQUIERDO (4)
Filosofo 2 -> Solto tenedores.

Filosofo 2 -> PENSANDO...
Filosofo 0 -> Solto tenedores.

Filosofo 0 -> PENSANDO...
Filosofo 4 -> Tomo tenedor DERECHO (0)
Filosofo 4 -> COMIENDO [tenedores 4 y 0]
Filosofo 1 -> Tomo tenedor IZQUIERDO (1)
Filosofo 1 -> Tomo tenedor DERECHO (2)
Filosofo 1 -> COMIENDO [tenedores 1 y 2]
Filosofo 3 -> Tomo tenedor IZQUIERDO (3)
Filosofo 4 -> Solto tenedores.

Filosofo 4 -> PENSANDO...
Filosofo 3 -> Tomo tenedor DERECHO (4)
```

Lfn. 22, col. 2 Espacios: 4 UTF-8 LF {} C Linux

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

Filosofo 4 -> PENSANDO...
Filosofo 3 -> Tomo tenedor DERECHO (4)
Filosofo 3 -> COMIENDO [tenedores 3 y 4]
Filosofo 0 -> Tomo tenedor IZQUIERDO (0)
Filosofo 1 -> Solto tenedores.

Filosofo 1 -> PENSANDO...
Filosofo 2 -> Tomo tenedor IZQUIERDO (2)
Filosofo 0 -> Tomo tenedor DERECHO (1)
Filosofo 0 -> COMIENDO [tenedores 0 y 1]
Filosofo 3 -> Solto tenedores.

Filosofo 3 -> PENSANDO...
Filosofo 0 -> Solto tenedores.

==> Filosofo 0 TERMINO sus 3 ciclos.
Filosofo 2 -> Tomo tenedor DERECHO (3)
Filosofo 2 -> COMIENDO [tenedores 2 y 3]
Filosofo 1 -> Tomo tenedor IZQUIERDO (1)
Filosofo 4 -> Tomo tenedor IZQUIERDO (4)
Filosofo 4 -> Tomo tenedor DERECHO (0)
Filosofo 4 -> COMIENDO [tenedores 4 y 0]
Filosofo 2 -> Solto tenedores.

Filosofo 1 -> Tomo tenedor DERECHO (2)
Filosofo 1 -> COMIENDO [tenedores 1 y 2]
==> Filosofo 2 TERMINO sus 3 ciclos.
Filosofo 4 -> Solto tenedores.

==> Filosofo 4 TERMINO sus 3 ciclos.
Filosofo 3 -> Tomo tenedor IZQUIERDO (3)
Filosofo 3 -> Tomo tenedor DERECHO (4)
Filosofo 3 -> COMIENDO [tenedores 3 y 4]
```

Lfn. 22, col. 2 Espacios: 4 UTF-8 LF {} C Linux

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

Filosofo 3 -> PENSANDO...
Filosofo 0 -> Solto tenedores.

==> Filosofo 0 TERMINO sus 3 ciclos.
Filosofo 2 -> Tomo tenedor DERECHO (3)
Filosofo 2 -> COMIENDO [tenedores 2 y 3]
Filosofo 1 -> Tomo tenedor IZQUIERDO (1)
Filosofo 4 -> Tomo tenedor IZQUIERDO (4)
Filosofo 4 -> Tomo tenedor DERECHO (0)
Filosofo 4 -> COMIENDO [tenedores 4 y 0]
Filosofo 2 -> Solto tenedores.

Filosofo 1 -> Tomo tenedor DERECHO (2)
Filosofo 1 -> COMIENDO [tenedores 1 y 2]
==> Filosofo 2 TERMINO sus 3 ciclos.
Filosofo 4 -> Solto tenedores.

==> Filosofo 4 TERMINO sus 3 ciclos.
Filosofo 3 -> Tomo tenedor IZQUIERDO (3)
Filosofo 3 -> Tomo tenedor DERECHO (4)
Filosofo 3 -> COMIENDO [tenedores 3 y 4]
Filosofo 1 -> Solto tenedores.

==> Filosofo 1 TERMINO sus 3 ciclos.
Filosofo 3 -> Solto tenedores.

==> Filosofo 3 TERMINO sus 3 ciclos.

=== Todos los filosofos comieron. Fin del programa. ===
[1] + Done "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microsoft-MIEngine-In-3od02grf.bcg" 1>"/tmp/Microsoft-MIEngine-Out-iicttmfb.npc"
juanj@Juanjo:~$
```